Assignment 6 - Observer and Visitor Patterns

Variant 15 - Public Transport

Student	Arnur Sadyrov
Group	SE-2532
Technology	Java 17, JUnit 5
Repository	https://github.com/qrnvr/Assignment-6-Observer-Visitor

Project Overview

The system combines Observer and Visitor in a public-transport domain. Vehicle-arrival and ridership sources publish immutable events to runtime subscribers. A delayed arrival is republished as a delay event, which causes an accessibility Visitor to run only on the affected route subtree. The result returns to the Observer side as an inspection-report event.

Observer

Two live sources are implemented: VehicleArrivalSource and RidershipSource. The project defines generic Subject<E> and Observer<E> interfaces. Subscribers register per event type and can subscribe or unsubscribe at runtime. OneShotArrivalListener unsubscribes from inside its own handler. RidershipAggregator is both an Observer and a Subject and republishes cumulative route metrics.

Visitor

The composite tree is TransportNetwork -> Route -> Stop -> Bus / Tram / StationFacility / TicketMachine. TransportVisitor<R> uses double dispatch. The four operations are TicketRevenueVisitor, AccessibilityAuditVisitor, AccessibleElementFilterVisitor, and a hand-written nested JsonExportVisitor.

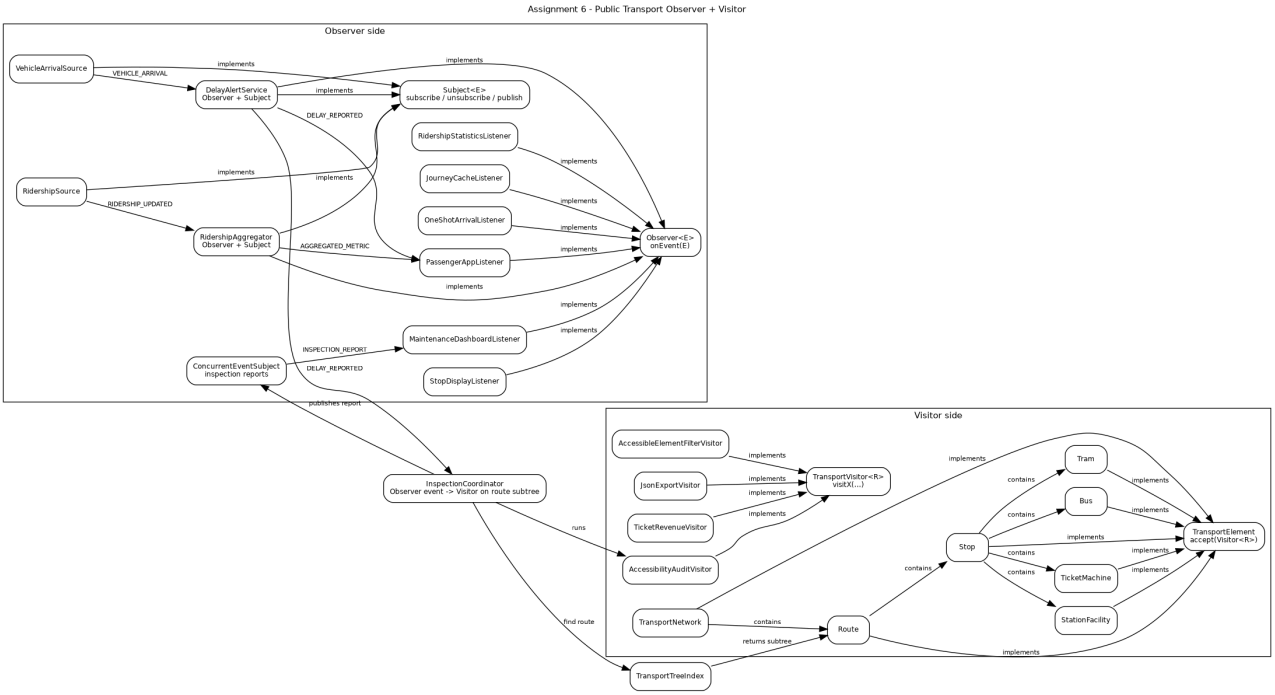
Concurrency and Tests

ConcurrentArrivalSimulation uses ExecutorService with a configurable pool size and at least three producer tasks. Listener registries use ConcurrentHashMap and CopyOnWriteArrayList; listener state uses concurrent collections, atomics, or LongAdder. The project contains 20 JUnit 5 tests, including the required 4-thread x 1,000-event stress test with an exact expected count of 4,000.

Trade-off Proof

A dedicated commit adds JsonExportVisitor without changing element classes. A separate commit adds TicketMachine and therefore updates the Visitor interface and all existing visitors. This demonstrates the classic Visitor trade-off: new operations are cheap when element types are stable, while new element types are more expensive.

UML Class Diagram



The integration link is InspectionCoordinator: it receives DELAY_REPORTED, resolves the matching Route, runs AccessibilityAuditVisitor on that subtree only, then publishes INSPECTION_REPORT.